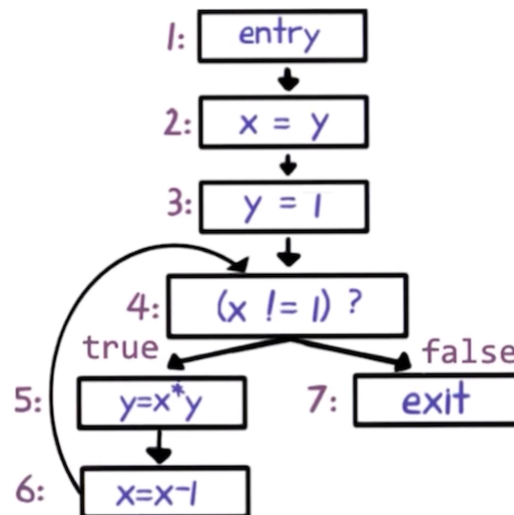


Ingeniería del Software II

Práctica #5 – Dataflow Analysis

Ejercicio 1

Sea el siguiente control-flow graph para una función:



Ejecutar el algoritmo caótico iterativo para el análisis de Reaching Definitions hasta alcanzar la estabilidad de los conjuntos *IN* y *OUT*. Completar la siguiente cuadro con el valor final de los conjuntos *IN* y *OUT*:

Nodo n	IN[n]	OUT[n]
1	-	$\emptyset$
2	$\emptyset$	$\{\langle x, 2 \rangle\}$
3	$\{\langle x, 2 \rangle\}$	$\{\langle x, 2 \rangle, \langle y, 3 \rangle\}$
4		
5		
6		
7		

Ejercicio 2

Sean la la función $Transfer(n, S)$ para el análisis de Reaching Definitions que matan y generan respectivamente la información de dataflow, completar las ecuaciones de dataflow que caracterizan los conjuntos *IN*[n] y *OUT*[n] para dicho análisis.

Ejercicio 3

Sea el siguiente programa, donde MASK, IA, IQ, IR, IM y AM son constantes.

```

float foo(int pid) {
1:  int i, j, h;
2:  i = pid ^ MASK;
3:  int k = i / IQ;

```

```

4:  j = IA * (i - k * IQ) - IR * k;
5:  h = j ^ MASK;
6:  if (h < 0)
7:    h = h + IM;
8:  float answer = AM * h;
9:  return answer * pid / k;
}

```

- Construir su control-flow graph.
- Computar el análisis Live Variables.
- ¿Qué pasaría si las constantes fueran variables?

Ejercicio 4

Computar los conjuntos IN y OUT para el análisis de Available Expressions. Considerar a $m[i]$ como una expresión.

```

void foo(int [] m) {
1:  int a = 3;
2:  int i = a + 2;
3:  while (i <= a) {
4:    int t = m[i]*a;
5:    m[i] = t;
6:    int j = i + 1;
7:    i = j;
8:    a = bar(m, a)
  }
}

```

Ejercicio 5

Categorizar los análisis dataflow Live Variables, Reaching Definitions, Very Busy Expressions y Available Expressions.

	<i>Forward</i>	<i>Backward</i>
<i>May</i>		
<i>Must</i>		

Ejercicio 6

Sea el siguiente programa:

```

1 void bar(int p1, int p2) {
2   int a = p1;
3   int b = p2;
4   int x = a+b;
5   int y = a*b;
6   while (y>a) {
7     a = a +1;
8     x = a+b;

```

```

9      }
10     return;
11    }

```

- Escribir el control-flow graph del programa
- Ejecutar el algoritmo caótico iterativo para el análisis de Live Variables hasta alcanzar la estabilidad de los conjuntos IN y OUT y completar la siguiente tabla:

Nodo n	IN[n]	OUT[n]
...	...	...

Ejercicio 7

Dado el siguiente programa y el análisis de *Available Expressions*:

```

1  int bar() {
2      int x = 3;
3      int y = 4;
4      int z = 0;
5      int j = 0;
6      while (j < 6) {
7          x = x + j;
8          if j % 2 == 0 {
9              y = y * 3;
10             } else {
11                 z = z + y;
12             }
13             j = j + 1;
14         }
15         int w = x + y;
16         return w * z;
17     }

```

- Construir el CFG del programa.
- Calcular el valor abstracto de las variables para los conjuntos IN y OUT de cada nodo en el CFG.

Ejercicio 8

Dado el siguiente programa y el análisis de *Very Busy Expressions*:

```

1  int foo() {
2      int a = 1;
3      int b = 2;
4      int c = 3;
5      int i = 0;
6      while (i < 5) {
7          a = a + b;
8          if (i % 2 == 0) {
9              b = b * c;
10             } else {
11                 c = a + b;

```

```
12         }
13         i = i + 1;
14     }
15     int d = a * b;
16     int e = b + c;
17     return d - e;
18 }
```

- a. Construir el CFG del programa.
- b. Calcular el valor abstracto de las expresiones para los conjuntos IN y OUT de cada nodo en el CFG, según el análisis de *Very Busy Expressions*.